Regularization techniques

Regularization in machine learning concerns the ways we try to avoid overfitting, or said otherwise, to avoid the model adapting too closely to patterns it finds in the training data that can't be generalized to new, unseen data.

L1 regularization

In both L1 and L2 regularization, the idea is to penalize large weights. If the model assigns a very high weight to one feature, it becomes over-reliant on that single signal and risks ignoring other features that might be equally relevant. By adding a penalty for large weights to the loss function, we force the model to spread its attention more evenly.

In L1, the penalty added to the loss is the sum of the absolute values of the weights. Since the model tries to minimize the loss, a large weight now directly increases what it needs to minimize, so it is pushed to reduce that weight. L1 can push certain weights all the way to zero, making it to ignore those features entirely.

L2 regularization

L2 works the same way, but adds the sum of the squared weights to the loss instead. Because the penalty grows with the square of the weight, large weights are penalized much more heavily than small ones: this creates a smoother, more gradual and proportional reduction.

Example: let’s imagine a model predicting if a picture holds a cat or a dog. The model starts recognizing a cat by its pointy, “standing” ears. But it gives such a huge weight to this feature, that a Doberman, ready to attack, with his ears pointed, is categorized as a cat. With L1, the model could consider pointy ears is too “heavy” and filter out this feature completely. Even though pointy ears can still be a good indication, so in this case L2 would be more appropriate; the weight is being pulled down, but not cancelled.

Dropout

With Dropout, during the learning stage, a certain percentage of neurons are randomly “turned off” at each batch. This means the same training data is processed by a slightly different version of the network each time. The model is forced to learn patterns that don't rely on any single neuron, which makes it more robust and less prone to overfitting.

Example: if we keep the example of the pointy ears; if the neurons responsible for the heavy weight of this feature get switched off from time to time, the model is forced to also look at other features.

Data Augmentation

The best way to reduce overfitting is more data, but data can be hard to come by. That's where data augmentation fits in: we can transform the data we already have to multiply it at a lesser cost.

Example: we can flip around the cat and dog pictures. The cat will still be a cat, and likely have pointy ears.

Early Stopping

At each epoch, the weights are being updated based upon the results of the training set. These weights are then tested on a second set of data (the dev set) where the loss is monitored. If the loss on the dev set stops improving for several consecutive epochs, we stop the training of the model and revert to the best epoch : the one where the loss was at its lowest.

Example: the training set has a lot of horizontal pictures of cats, and vertical pictures of dogs. For no reason at all, total odds. In the training, the model starts to see this patterns, and gives more and more weight to the form of the picture to make its decision. That's where some independent data could come in, data the model doesn't use to train, and that would tell the model it's on a wrong track.